

La Computación Paralela en el Aula y en la Profesión

Resumen y notas del show

Luis Francisco Padilla

Estuardo Castro

26 de septiembre de 2026

Índice

1. Datos del episodio	1
2. Descripción del episodio	2
3. Capítulos	2
4. Resumen de la conversación	3
4.1. El rol del Machine Learning Engineer	3
4.2. Métricas del sistema y métricas del modelo	3
4.3. Colas de tareas y workers	3
4.4. Los límites del paralelismo y la ley de Amdahl	4
4.5. Optimización en demos, pruebas de concepto y producción	4
4.6. Costos y decisiones de arquitectura paralela	4
4.7. Decisiones fundamentadas y reglas del negocio	4
4.8. Paralelización en visión por computadora	5
4.9. El futuro del paralelismo	5
4.10. Cierre	5
5. Preguntas discutidas	5
6. Conceptos clave mencionados	6
7. Herramientas y tecnologías mencionadas	6
8. Referencias y recursos adicionales	7

1. Datos del episodio

Campo	Detalle
Programa	Podcast del curso Computación Paralela y Distribuida, sección 20
Institución	Universidad del Valle de Guatemala, Facultad de Ingeniería

Campo	Detalle
Presentadores	Estuardo Castro y Luis Francisco Padilla
Invitado	Daniel Valdez, exalumno de UVG y Machine Learning Engineer en Improgress
Modalidad	Entrevista virtual
Duración	45 minutos aprox. (de la introducción de Estuardo a la frase final del invitado)
Fecha de grabación	26 de septiembre de 2026

2. Descripción del episodio

Daniel Valdez se graduó hace poco de Ingeniería en Ciencias de la Computación y Tecnologías de la Información en la Universidad del Valle de Guatemala. Hoy trabaja como *Machine Learning Engineer* en Improgress, donde construye automatizaciones y productos para clientes. En este episodio nos cuenta qué hace un ML Engineer y por qué es el puente entre el *data scientist* y el *backend*. También explica cómo el paralelismo y la computación distribuida le ayudan a cumplir tiempos de respuesta, manejar recursos y escalar sistemas reales.

Hablamos de colas de tareas y *workers*, de los límites que impone la ley de Amdahl, de cuándo sí vale la pena optimizar y cuándo no, y de cómo traducir una decisión técnica a las reglas del negocio. Daniel también nos cuenta sobre un proyecto de visión por computadora que tiene que pasar de 5 a 50 cámaras. Cerramos con su visión del futuro: asignación dinámica de tareas con apoyo de inteligencia artificial y el posible impacto de la computación cuántica.

3. Capítulos

El episodio empieza con la introducción de Estuardo y termina con la frase final de Daniel, cuando se despiden y se desconecta. Los tiempos se cuentan desde el inicio de la introducción.

Tiempo	Tema
00:00	Introducción y presentación de los anfitriones
01:25	Presentación de Daniel Valdez
02:11	Qué hace un Machine Learning Engineer
07:27	Analogía: el software como un carro
09:40	Métricas del sistema frente a métricas del modelo
12:00	Paralelismo, colas de tareas y <i>workers</i> en APIs
14:00	Técnicas para paralelizar y el problema del <i>overhead</i>
16:33	Ley de Amdahl y el cuello de botella secuencial
18:13	Optimización en demos y pruebas de concepto
20:02	Sistemas de baja latencia: el caso de Spotify
22:08	Balancear costos, complejidad y funcionalidad
25:22	Condiciones de carrera, memoria compartida y costos en la nube

Tiempo	Tema
26:35	Decisiones fundamentadas y comunicación con <i>stakeholders</i>
29:15	Reglas de negocio: optimización de efectivo en cajeros y agencias
33:37	Tecnologías de paralelización más allá de PyTorch
34:33	Proyecto de visión por computadora: de 5 a 50 cámaras
37:42	El futuro del paralelismo
42:03	Cierre y agradecimientos
45:33	Mensaje final del invitado

4. Resumen de la conversación

4.1. El rol del Machine Learning Engineer

Daniel explica que el ML Engineer lleva un modelo de inteligencia artificial desde que se diseña hasta que funciona en producción. El *data scientist* diseña el modelo y el *backend* lo consume, y el ML Engineer une a los dos: entiende el modelo aunque no lo diseñe, encuentra la forma de servirlo y lo mantiene para que el *backend* lo use. También cuida el ciclo completo de los datos y define métricas para confirmar que el modelo sigue funcionando.

Luis cuenta que trabajó en equipos sin esta figura, donde recibía el modelo “en crudo”, y que eso le causó problemas para alimentarlo y servirlo. Luego propone una analogía: el software es un carro. El modelo es el motor, el *frontend* es la carrocería visible, el *backend* es el resto del chasis y el ML Engineer es la caja de cambios que ajusta el motor al carro. La caja puede ser “manual”, cuando el modelo pesado se corre una vez y el servidor solo entrega resultados ya calculados, o “automática”, cuando un modelo ligero se ejecuta en vivo con cada solicitud.

4.2. Métricas del sistema y métricas del modelo

Las métricas de un sistema son el rendimiento, el tiempo de respuesta, la carga de solicitudes simultáneas y el tráfico de usuarios. Un modelo tiene esas mismas métricas y además las suyas propias: calidad de las predicciones, sobreajuste, capacidad de adaptarse a datos nuevos y detección de cuándo deja de predecir lo que debe. Muchas de estas métricas son KPIs del negocio. Aun así, el ML Engineer no puede olvidar las métricas del sistema, y ahí es donde entran el paralelismo y la computación distribuida.

4.3. Colas de tareas y workers

Algunos modelos responden en milisegundos, pero otros procesos tardan minutos, como generar un archivo que toma cinco minutos. Frameworks como FastAPI suelen atender las solicitudes en un solo hilo principal, y bloquear ese hilo con una tarea larga detiene todo el sistema. La solución que describe Daniel es mandar esas tareas a una **cola de tareas** que atienden **workers** en segundo plano. Así la API sigue respondiendo mientras el trabajo pesado se procesa aparte, en paralelo si hace falta.

4.4. Los límites del paralelismo y la ley de Amdahl

Estuardo pregunta qué técnicas usa Daniel para que los procesos corran lo más rápido posible. Daniel responde que no hay fórmula mágica: meter 50 hilos no hace que algo tarde un milisegundo. Cada hilo consume recursos, y pasado cierto punto agregar más solo suma *overhead*. En los *pipelines* de limpieza y verificación de datos a veces sí conviene agregar hilos, pero en otros sistemas la secuencia de los datos no deja paralelizar más.

Estuardo lo relaciona con la **ley de Amdahl**: la parte secuencial de un programa no se puede paralelizar y pone el techo a la aceleración que se puede lograr, sin importar cuántos hilos se agreguen. Daniel está de acuerdo y agrega que en la práctica pasa algo parecido con el tiempo del equipo, porque no siempre alcanza para hacer una paralelización bien hecha.

4.5. Optimización en demos, pruebas de concepto y producción

En demos y pruebas de concepto, la optimización suele quedar en segundo plano. Las usan entre una y diez personas y lo que importa es mostrar que la idea funciona. Luis pregunta si la optimización debe verse como algo opcional o si alguna vez fue lo que le dio valor al producto. Daniel responde que todavía no le ha tocado un caso en que la paralelización fuera el diferenciador de una demo.

En producción la historia cambia. Los servicios de *streaming* como YouTube y Spotify necesitan el menor tiempo de respuesta posible. Daniel recuerda una anécdota de sus clases: a los ingenieros de Spotify les pedían que la canción empezara a sonar en el instante en que se presiona el botón. Cuando se diseña un sistema de verdad, se hacen diagramas de arquitectura y de topología, y la paralelización se planifica en serio.

Aun así, optimizar desde el principio a veces sí compensa. Si una prueba de concepto descarga y procesa muchos datos, paralelizar desde el inicio evita pasar una o dos horas esperando a que termine el proceso.

4.6. Costos y decisiones de arquitectura paralela

Luis conecta las decisiones de paralelización con los conceptos del curso: condiciones de carrera, memoria compartida y paso de mensajes. Una mala decisión cuesta microsegundos por proceso, y con millones de procesos eso se vuelve más tiempo de servidor y una factura más alta en AWS. Daniel concluye que el paralelismo se decide junto con los recursos disponibles y las reglas del negocio, y que la combinación nunca es perfecta.

4.7. Decisiones fundamentadas y reglas del negocio

Daniel insiste en que toda decisión sobre paralelismo tiene que estar respaldada con datos, no con “me lo dijo Codex” o “Gemini le dio el visto bueno”. Esto importa más cuando se negocia con *stakeholders* o con el PM, que muchas veces no ven la complejidad técnica. Si quieren X y no saben que eso obliga a sacrificar B, hay que explicárselo en términos que entiendan. Si no, terminan diciendo “hazlo como quieras” y cualquier falla cae sobre el ingeniero.

Como ejemplo cuenta un caso de Improgress: optimizar el efectivo en cajeros automáticos. El cliente quiere que los cajeros nunca se queden sin dinero, pero cada punto de precisión extra obliga a asignar más dinero a todos los cajeros. Luis agrega que en las agencias bancarias el objetivo es el contrario, que no sobre efectivo. Daniel lo compara con el **problema de la mochila**: hay que repartir recursos limitados entre objetivos que compiten. La lección es que las reglas del negocio guían el diseño del

producto y de la paralelización. Por eso hay que acordar criterios claros de cuándo el trabajo está terminado, sabiendo que nunca va a ser 100 % perfecto.

4.8. Paralelización en visión por computadora

Estuardo supone que, siendo de *machine learning*, Daniel paraleliza sobre todo con PyTorch, y le pregunta qué otras herramientas usa. Daniel habla de un proyecto actual: un sistema de nodos de cámaras que hoy tiene 5 y que debe llegar a 50. Para lograrlo combina varias herramientas:

- El **SDK de cada cámara** para capturar las imágenes.
- **Hilos**, tanto de Python como de sistema (Pthreads en C).
- **Workers** y una **cola de trabajo**: un grupo fijo de hilos que siempre está activo, y las tareas se les van asignando conforme llegan. Daniel no recordaba el nombre de la técnica; es el patrón conocido como *thread pool*.
- **Subprocesos** asignados a cada cámara.
- **Semáforos** para controlar el acceso concurrente y evitar condiciones de carrera.

El requisito principal del cliente es mantener una latencia baja aunque crezca el número de cámaras.

4.9. El futuro del paralelismo

Para Daniel, el paralelismo siempre va a ser necesario porque es una de las mejores formas de mejorar tiempos de carga y de respuesta. El *machine learning* también depende de él para cumplir los KPIs de tiempo que pide el negocio. Ve dos tendencias hacia adelante:

1. **Paralelización asistida por IA**: sistemas que asignen procesos y tareas de forma dinámica y aprendan de la carga, en lugar de depender solo de heurísticas fijas.
2. **Computación cuántica**: todavía es experimental, pero trabaja con compuertas lógicas cuánticas y funciones probabilísticas. Muchos problemas que hoy se atacan con paralelismo clásico podrían resolverse de otra forma.

Luis agrega que una computadora que entienda sus propias cargas podría repartir el trabajo para reducir los problemas de comunicación, las condiciones de carrera y el *overhead* del paso de mensajes. Si la computación cuántica se comercializa, cambiarían las reglas con que se diseñan los sistemas, algo que ve a la vez como emocionante y como un reto de adaptación.

4.10. Cierre

Los anfitriones agradecen a Daniel y le desean éxito con el proyecto de las cámaras. Él les desea lo mejor en su carrera como ingenieros y deja un mensaje final: “*Con vista al futuro y con vista al mañana.*”

5. Preguntas discutidas

1. ¿Quién eres y a qué te dedicas actualmente?
2. ¿Qué hace un Machine Learning Engineer y en qué se diferencia del *data scientist* y del *backend*?
3. ¿Qué técnicas usas en tu trabajo para paralelizar procesos y reducir el *delay*?
4. ¿Coincide lo que describes con la ley de Amdahl, donde la parte secuencial se vuelve el cuello de botella?

5. ¿Está bien dejar la optimización como algo opcional en una demo, o alguna vez la paralelización fue lo que le dio valor al producto?
6. Además de PyTorch, ¿qué otras librerías o tecnologías usas para paralelizar el manejo de datos?
7. ¿Cómo ves el futuro del *machine learning* y del paralelismo, sobre todo para quienes están por graduarse?
8. ¿Algún mensaje final para quienes nos escuchan?

6. Conceptos clave mencionados

Concepto	Cómo apareció en la conversación
Paralelismo y computación distribuida	Herramientas para cumplir latencia y administrar recursos
Cola de tareas y <i>workers</i> <i>Thread pool</i>	Sacar tareas largas del hilo principal de una API Grupo de hilos persistentes con asignación dinámica de tareas en el sistema de cámaras
<i>Overhead</i>	Costo de recursos que aparece al agregar hilos sin necesidad
Ley de Amdahl	La parte secuencial limita la aceleración que se puede lograr
Condiciones de carrera	Riesgo al procesar muchas cámaras al mismo tiempo
Semáforos	Mecanismo para sincronizar y evitar condiciones de carrera
Memoria compartida y paso de mensajes	Modelos de comunicación que hay que balancear al diseñar
Latencia	Requisito principal en <i>streaming</i> y en visión por computadora
Problema de la mochila	Analogía para repartir recursos entre objetivos que compiten
Computación cuántica	Posible cambio en la forma de resolver problemas paralelos

7. Herramientas y tecnologías mencionadas

- **PyTorch**: framework de aprendizaje profundo con soporte para GPU y paralelismo de datos.
- **FastAPI**: framework web de Python para servir modelos.
- **Hilos de Python** (*threading*) y **subprocesos**.
- **Pthreads**: hilos POSIX en C.
- **Semáforos**: primitiva de sincronización.
- **SDK de cámaras**: kits de desarrollo de cada fabricante para la captura de video.
- **AWS**: infraestructura en la nube cuyo costo depende del tiempo de cómputo.
- **Codex y Gemini**: asistentes de IA, mencionados como ejemplo de lo que *no* basta para justificar una decisión técnica.

8. Referencias y recursos adicionales

- Amdahl, G. M. (1967). *Validity of the single processor approach to achieving large scale computing capabilities*. AFIPS Spring Joint Computer Conference, pp. 483–485.
- Documentación de PyTorch sobre entrenamiento distribuido: https://pytorch.org/tutorials/beginner/dist_overview.html
- Documentación de FastAPI sobre tareas en segundo plano: <https://fastapi.tiangolo.com/tutorial/background-tasks/>
- Documentación del módulo `threading` de Python: <https://docs.python.org/3/library/threading.html>
- Documentación de `concurrent.futures`, que implementa *thread pools* en Python: <https://docs.python.org/3/library/concurrent.futures.html>
- Barney, B. *POSIX Threads Programming*. Lawrence Livermore National Laboratory: <https://hpc-tutorials.llnl.gov/posix/>
- Pacheco, P. y Malensek, M. (2021). *An Introduction to Parallel Programming* (2.^a ed.). Morgan Kaufmann.